

Resolving SWE-bench Verified Issues with a Local Coding Agent

INTRODUCTION

An open-weight model, with Claude Code as the harness, running on a laptop, resolved **10 SWE-bench Verified issues out of 13** it attempted. The official SWE-bench harness graded every patch against the benchmark’s hidden tests. No closed-source or cloud model took part.

MODEL CHOICE

The machine is a MacBook Pro with an M4 Pro (14 CPU cores, 20 GPU cores), 24 GB of unified memory and 273 GB/s of memory bandwidth, of which macOS gives the GPU about 16 to 18 GB. I chose **Ornith-1.5-35B-A3B**, which is a mixture of experts model that activates 3B of its 35B parameters per token. Its developers report 79% accuracy on the SWE-bench Verified dataset, which was the highest accuracy I found for a model that fits in 24 GB memory of my laptop (Qwen3.6-35B-A3B reports 73.4%, Devstral Small 2 reports 68%). Ornith, quantized to the 16.5 GB APEX Compact GGUF (about 3.8 bits per weight), generates 57 tokens/s here and fits on the GPU with a 64K context.

ARCHITECTURE

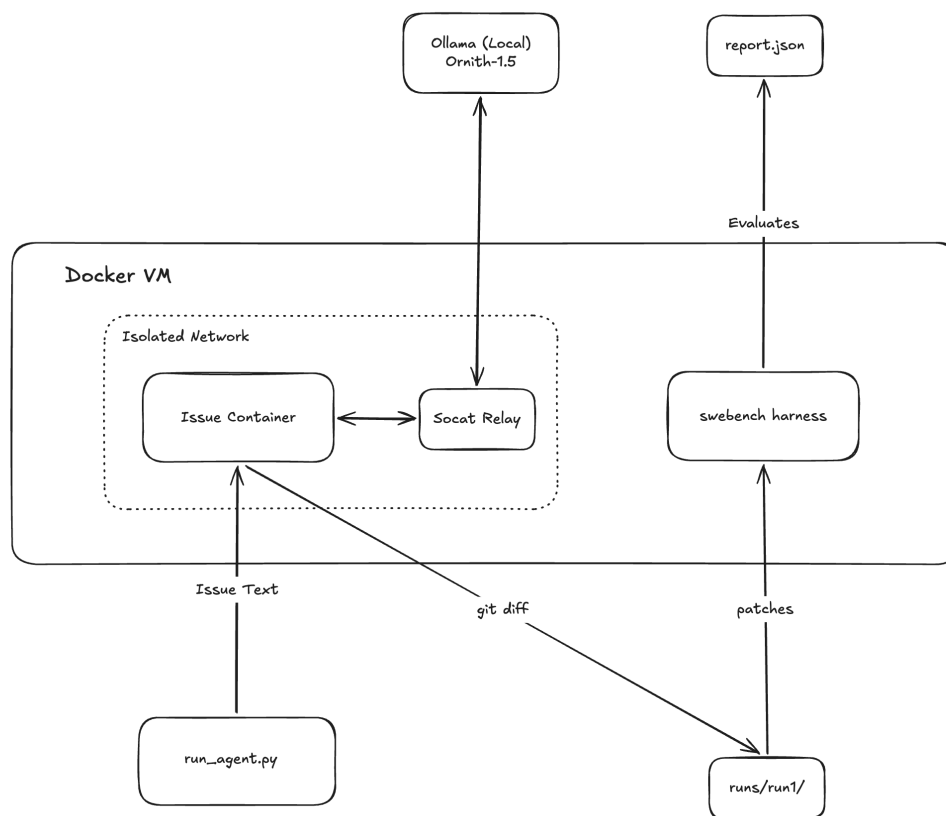


Figure 1: How one issue is attempted and graded.

Ollama serves the local Ornith-1.5 model through an Anthropic compatible API, so Claude Code can connect to it as a harness without any changes. Grading one issue works like this (Figure 1):

1. `run_agent.py` starts the issue’s official SWE-bench container, which holds the repository at the issue’s commit plus includes all its dependencies, and sends the issue text to Claude Code inside it so it can work on solving it.
2. Claude Code (connected to our local model via Ollama) works on the fix. Its only network access is through the socat relay to Ollama, the reason for that is to prevent the model from “cheating” by fixing an issue by just downloading the latest version of the library that contains the fix, and not writing any code to solve the issue.
3. When Claude Code stops, its code changes are saved to the `run1` folder.
4. The swebench harness applies each change in a fresh container and runs the hidden tests. It writes `report.json`, which marks the issue resolved only if all the tests pass.

DESIGN DECISIONS

The agent runs inside the task container. The container has the right Python and dependencies, so the agent can run Django’s tests, and it is the sandbox so the agent can do any action freely without the risk of doing something malicious on the system.

The agent has no internet. Agent containers sit on an internal Docker network that has no internet access. I added this mid run, after the agent on an issue ran `pip download django==3.0` “to find the actual upstream fix” rather than fixing it itself. The download timed out, and I discarded that attempt and reran the issue isolated.

A seed chose the issues. I sampled 10 from the 16 Django 3.0 issues labelled “<15 min fix”. After 7 of those 10 were resolved, I tried the rest of the pool and stopped at 10 resolved.

RESULTS

Issue	Resolved	Hidden tests	Existing tests	Minutes	Turns
10880 DISTINCT with a condition	yes	1/1	55/55	3.1	21
10914 default upload permissions	yes	1/1	98/98	3.5	28
10999 negative durations	no	0/2	10/10	6.1	19
11066 content type saved to wrong DB	yes	1/1	3/3	1.5	7
11099 trailing newline in usernames	yes	3/3	19/19	1.7	15
11179 delete() keeps the PK	yes	1/1	40/40	3.0	19
11433 cleaned_data vs model defaults	no	0/1	138/142	18.3	43
11451 authenticate() with no password	no	0/6	45/45	4.6	26
11490 values() on combined queries	yes	1/1	23/23	11.7	43
11603 DISTINCT for Avg and Sum	yes	2/2	58/58	5.6	36
11239 Postgres client certificates	yes	1/1	5/5	3.4	16
11133 memoryview in HttpResponse	yes	1/1	64/64	5.7	22
11163 model_to_dict(fields=[])	yes	1/1	141/141	1.8	17

The first 10 issues, fixed before any run, gave 7 of 10, and the three extras all resolved. The agent spent 70 minutes on all 13, a median of 3.5 minutes per issue.

ANALYSIS

Every run followed the same loop. The agent found the code, read it, reproduced the bug with a short script, edited, and ran Django’s test suite, 2 to 11 times per issue. No run edited a test file or ended with an empty patch. In the five resolved issues I compared line by line with Django’s own fix, the agent changed the same line in the same or an equivalent way.

All three failures fixed the example and missed a neighbouring case. On 10999 (the issue of negative durations) the agent applied the one-line regex change the issue text suggests, while Django’s fix moves the sign into its own group, so the tests for negative and PostgreSQL-style durations still failed. On 11451 (the authenticate with no password issue) it returns early when the username is missing but still queries the database when the password is missing, so all 6 hidden tests fail. On 11433 (the cleaned_data vs model defaults) it deleted the check that keeps model defaults instead of narrowing it, which broke 4 existing tests. The agent saw those failures in its own test runs and dismissed them as “old behavior we’re intentionally changing”. That run was also the only one past the 64K context (67K tokens). Failed runs took longer (median 6.1 minutes against 3.2) and used more context (mean largest prompt 43K tokens against 28K).

The agent tried to look up answers. Nothing in the prompt suggested it, yet on 11433 (the issue of cleaned_data vs model defaults) the agent tried twice to fetch the upstream fix: `pip download django==3.0`. Without network isolation, a score from this setup could include copied fixes i.e. the agent would have just updated the django version without actually writing any code.

Memory of the machine was a constraint. The model (17 GB) and Docker’s VM (8 GB) together exceed 24 GB, and macOS swapped up to 18.9 GB. Swap cost speed but changed no result.

Limitations. Thirteen easy issues from one repository are a small sample to test the performance of the model comprehensively. Each issue got one attempt at temperature 1.0, so reruns could differ. The model ran at about 3.8 bits per weight in a different harness than the one which gave the 79% score, so the two numbers are not directly comparable.

Code: `runner/` (runner, grading, summary), `bin/claude-local` (host launcher), `runs/run1/` (transcripts, patches, results), `logs/run_evaluation/` (harness reports).